In [1]:
```
# Behzad Ghabaei
# CS 82A - Data Science
# Text Book Prolem 5.3
# Housing type bar chart
# module5_lab2.ipynb
# custdata.tsv
# Instructor Seno
# 9/30/2026
# 10/19/2026
```

In [2]:
```
import pandas as pd
df = pd.read_csv("custdata.tsv", sep="\t")
```

We first would like to look at the dataset with tools like: describe(), info(), shape, head(), and columns.

In [3]:
```
df.describe()
```

Out[3]:

	custid	income	num.vehicles	age
count	1.000000e+03	1000.000000	944.000000	1000.000000
mean	6.984997e+05	53504.771000	1.916314	51.699815
std	4.135083e+05	65478.065729	1.101618	18.863433
min	2.068000e+03	-8700.000000	0.000000	0.000000
25%	3.456668e+05	14600.000000	1.000000	38.000000
50%	6.934030e+05	35000.000000	2.000000	50.000000
75%	1.044606e+06	67000.000000	2.000000	64.000000
max	1.414286e+06	615000.000000	6.000000	146.680197

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   custid        1000 non-null   int64
 1   sex           1000 non-null   str
 2   is.employed   672 non-null    object
 3   income        1000 non-null   int64
 4   marital.stat  1000 non-null   str
 5   health.ins    1000 non-null   bool
 6   housing.type  944 non-null    str
 7   recent.move   944 non-null    object
 8   num.vehicles  944 non-null    float64
 9   age           1000 non-null   float64
 10  state.of.res  1000 non-null   str
dtypes: bool(1), float64(2), int64(2), object(2), str(4)
memory usage: 79.2+ KB
```

In [5]:
```
df.shape
```

Out[5]:
```
(1000, 11)
```

In [6]:
```
df.head()
```

Out[6]:

	custid	sex	is.employed	income	marital.stat	health.ins	housing.type	recent.move	num.vehicles	age
0	2068	F	NaN	11300	Married	True	Homeowner free and clear	False	2.0	49.0
1	2073	F	NaN	0	Married	True	Rented	True	3.0	40.0
2	2848	M	True	4500	Never Married	False	Rented	True	3.0	22.0
3	5641	M	True	20000	Never Married	False	Occupied with no rent	False	0.0	22.0
4	6369	F	True	12000	Never Married	True	Rented	True	1.0	31.0

In [7]:
```
df.columns
```

Out[7]:
```
Index(['custid', 'sex', 'is.employed', 'income', 'marital.stat', 'health.ins',
       'housing.type', 'recent.move', 'num.vehicles', 'age', 'state.of.res'],
      dtype='str')
```

Dropping the NaN entries but first observing .isna.sum().

In [8]:
```
df["housing.type"].isna().sum()
```

Out[8]:
```
np.int64(56)
```

We see that **56** housing type rows are missing. This will show us how many are missing after dropping NaN. How many rows in housing type are missing values? 1000 - 944 = 56.

The dropna function can be used to get rid of NaN.

In [9]:
```
# Save the cleaned data back into df
df = df.dropna(subset=["housing.type"])
```

example of using income to fill in housing type, not used here

```
def housing_type_condition(v):
    if v < 10000:
        return "Occupied with no rent"
    elif v < 50000:
        return "Rented"
    elif v < 100000:
        return "Homeowner with mortgage/loan"
    else:
        return "Homeowner free and clear"

# Pass the 'income' value row-by-row using axis=1
df['housing.type'] = df.apply(lambda row: housing_type_condition(row['income']),
axis=1)
```

In [10]:
```
df["housing.type"].isna().sum()
```

Out[10]:
```
np.int64(0)
```

Taking a look at each column and missing values.

In [11]:
```
df.isna().sum()
```

Out[11]:
```
custid            0
sex               0
is.employed     280
income            0
marital.stat      0
health.ins        0
housing.type      0
recent.move       0
num.vehicles      0
age               0
state.of.res      0
dtype: int64
```

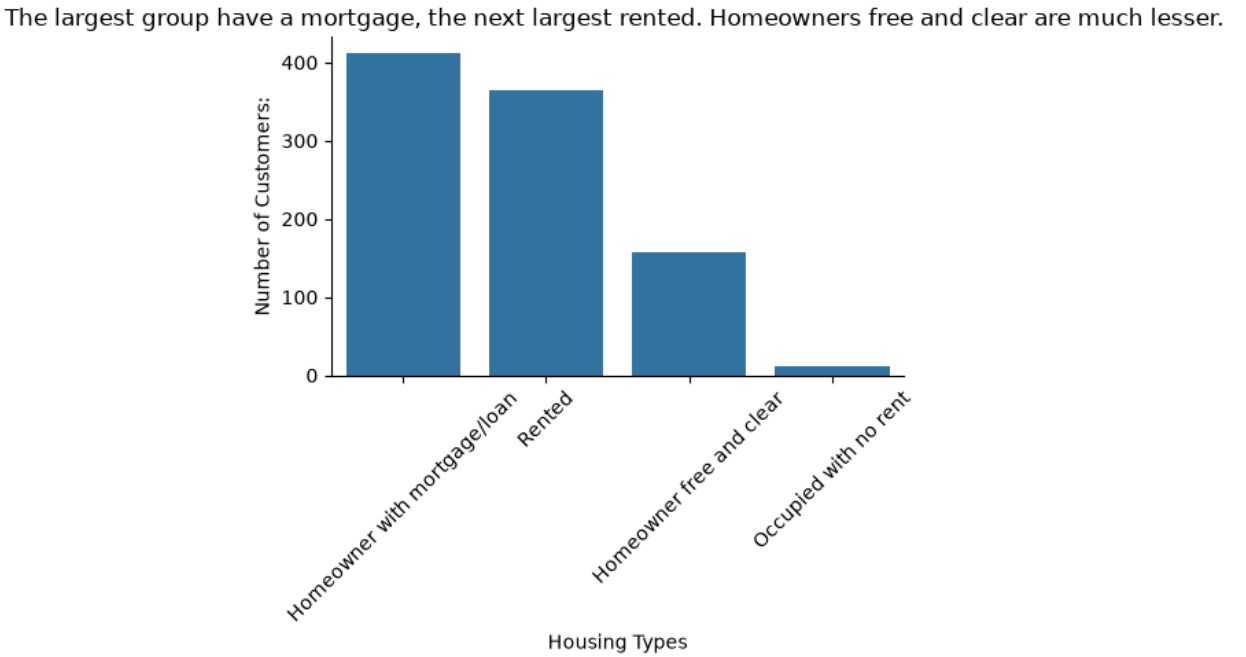
We have made two barplots with different libraries one with seaborn and the other with matplotlib.pyplot library.

Seaborn can plot the data from a column against the count of rows. Catplot demos are useful to observe barplot comparisons.

In [12]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.catplot( data=df, x="housing.type", kind="count", order= df["housing.type"].value_counts().index

plt.title("The largest group have a mortgage, the next largest rented. Homeowners free and clear are
plt.xlabel("Housing Types")
plt.ylabel("Number of Customers:")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



The takeaway is that most cutomers have a mortgage, and the next largest group are renting, the remaining customers, lesser in size, actually own the home, and a very small group are excused from payments.

The next bar plot example does the same thing and returns the same results, first import the matplotlib.pyplot library.

In [13]:
```
import matplotlib.pyplot as plt
```

Count how many rows (customers) belong to each housing type with two functions.

value_counts(): loops through that column, groups the identical values together (like "Rented", "Homeowner with mortgage/loan") and counts how many rows (customers) belong to each group.

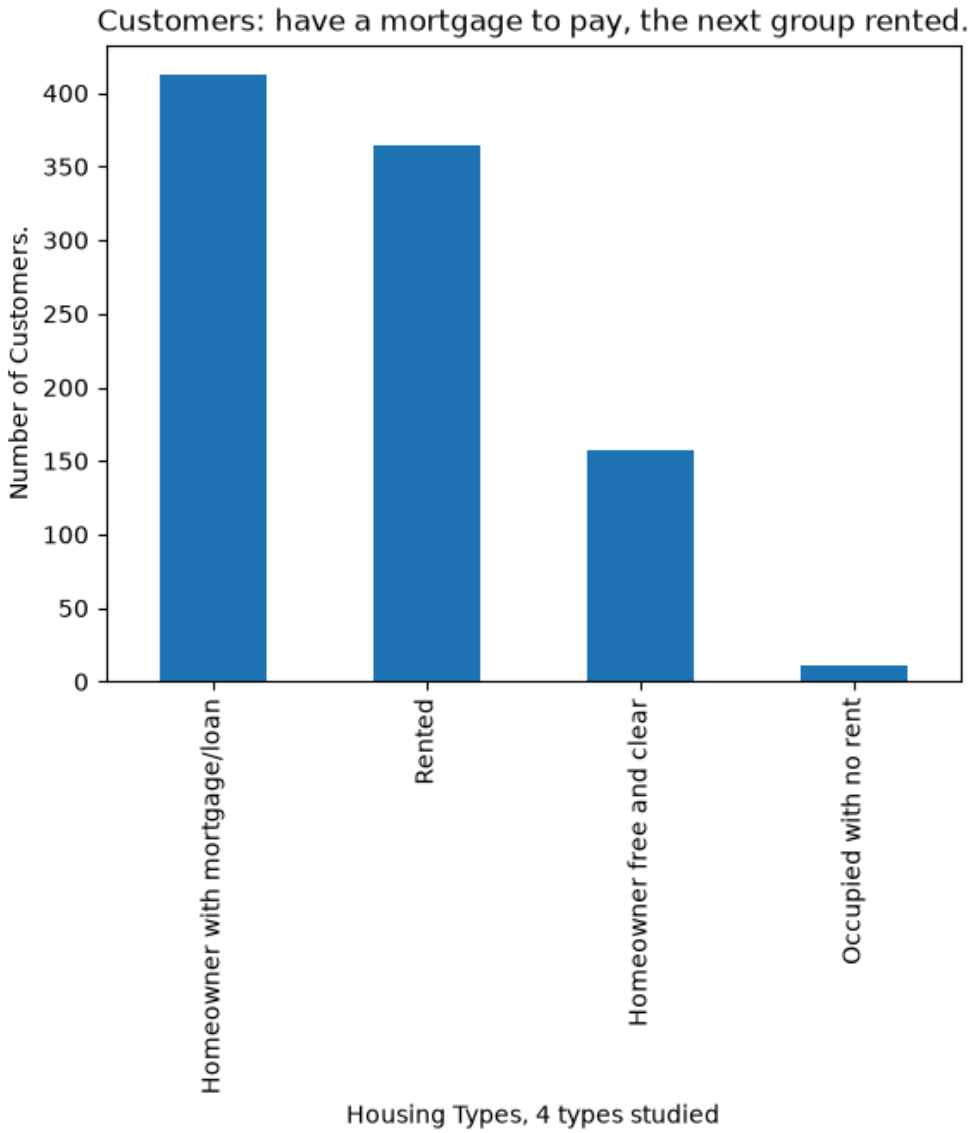
sort_values(): sorted by number of customers, sorts without altering the underlying data.

In [14]:
```
housing_counts = df["housing.type"].value_counts().sort_values(ascending=False) #sort_index()
print("housing count", housing_counts)
```

```
housing count housing.type
Homeowner with mortgage/loan    412
Rented                          364
Homeowner free and clear        157
Occupied with no rent            11
Name: count, dtype: int64
```

The bar plot uses number of customers and housing type, with labels and title including the 4 second takeaway.

In [15]:
```
housing_counts.plot(kind="bar")
plt.title("Customers: have a mortgage to pay, the next group rented.")
plt.xlabel("Housing Types, 4 types studied")
plt.ylabel("Number of Customers.")
#plt.xticks(rotation=45)
plt.show()
```



The takeaway is that the participants either pay a mortgage plan or a landlord. The small remainder are done paying for their property. The smallest group actually occupy with no rent.

We can also list the number of rows excluded.

In [16]:
```
print(f"Number of rows excluded: {1000 - len(df)}")
```

```
Number of rows excluded: 56
```

Out of 944 customers, most have a mortgage or rent. Only missing housing.type rows dropped. The bar plot shows that home owners with a mortgage are the predominant customers, renting is the next largest group and next, smaller in size, are the occupants that are homeowners free and clear, trailing as the smallest group are the people who occupied housing with no rent. (seabrn.pydata.org | Seaborn Barplots: Catplot | https://seaborn.pydata.org/tutorial/categorical.html#bar-plots)

Behzad Ghabaei - End Of Lab